# Labo 17 — Helm en oplevering

*Theorie — hoe een stack van elf manifesten een geversioneerd pakket wordt dat je met één commando installeert, bijwerkt en terugdraait; hoe een image van de CI naar de cluster reist; en wat er verandert wanneer de cluster niet langer je eigen k3s is.*

## Doelstellingen

- Een **Helm chart** lezen: `Chart.yaml`, `values.yaml`, `templates/`; *release* en *revision* begrijpen.
- Installeren, bijwerken, terugdraaien: `helm install / upgrade / rollback / history / uninstall`.
- Hetzelfde doen zonder templates met **Kustomize** (base + overlays) en weten wanneer je wat kiest.
- Een **strategie voor tags en digests** aannemen, en k3s laten pullen vanuit een registry.
- De keten **CI/CD → cluster**, **GitOps**, en k3s tegenover EKS/AKS/GKE/OpenShift plaatsen.

---

## 1. Van losse YAML naar een pakket

In labo 16 bestond de stack uit een tiental manifesten: Deployments, Services, ConfigMaps, Secret, StatefulSet, Ingress, LimitRange. Om ze in test en daarna in productie op te leveren, zou je ze moeten kopiëren en het aantal replica's, de image-tag, het Spring-profiel, de host van de Ingress… aanpassen in gedupliceerde bestanden. En wanneer versie 1.1 kapot blijkt, met de hand terugzoeken wat er eerder stond. Dat is precies het probleem dat `compose.override.yaml` in labo 09 oploste voor één machine. Kubernetes heeft twee antwoorden, die elkaar aanvullen: **Helm** (templates en waarden) en **Kustomize** (patches op zuivere YAML).

> **→ DOCKER-HERHALING**
> `compose.yaml` + `compose.prod.yaml` ↔ `values.yaml` + `values-prod.yaml`; `podman compose up` ↔ `helm install`; `podman compose down` ↔ `helm uninstall` (dat, net als `down` zonder `-v`, de volumes bewaart); `podman kube play` (labo 11) installeerde ook een YAML, maar zonder geschiedenis of waarden. De tags en digests van labo 10 blijven ongewijzigd: het zijn dezelfde images, in dezelfde registry `localhost:5000`.

## 2. Helm: chart, release, revision

Een **chart** is een map (of een `.tgz`-archief):

```
stack/
├── Chart.yaml          # naam, versie van de chart, appVersion
├── values.yaml         # alle standaardwaarden — wat per omgeving verschilt
├── values-prod.yaml    # overschrijvingen voor één omgeving (alleen de verschillen)
├── templates/          # de manifesten, met gaten: {{ .Values.api.replicas }}
│   ├── api.yaml  web.yaml  db.yaml  configmap.yaml  secret.yaml  ingress.yaml
│   ├── _helpers.tpl    # herbruikbare stukken (gemeenschappelijke labels)
│   └── NOTES.txt       # tekst die na de installatie getoond wordt
└── files/              # bestanden die ongewijzigd ingevoegd worden: {{ .Files.Get
"files/nginx-proxy.conf" }}
```

De templates zijn **Go templates**: `{{ .Values.api.image.tag }}`, `{{ .Release.Name }}`, `{{ if .Values.web.ingress.enabled }}`, `{{ toYaml .Values.api.resources | nindent 12 }}`. `helm template` rendert de YAML zonder iets te installeren; `helm lint` controleert de chart. Een **release** is een chart die onder een naam in een namespace geïnstalleerd is; elke `install`, `upgrade` of `rollback` maakt een **revision**, die Helm opslaat in een Secret `sh.helm.release.v1.<naam>.v<n>` van de namespace: dat maakt `helm history` en `helm rollback` mogelijk zonder server.

| Commando | Effect |
|---|---|
| `helm install stack ./stack -n lab17 --create-namespace` | revision 1; toont `NOTES.txt` |
| `helm upgrade stack ./stack --set api.replicas=3 -f values-prod.yaml` | volgende revision; `--reuse-values` behoudt de vorige `--set`'s |
| `helm rollback stack 1` | nieuwe revision "Rollback to 1" (je gaat niet terug, je gaat vooruit) |
| `helm history` / `helm get values` / `helm get manifest` | wat geïnstalleerd is, met welke waarden |
| `helm uninstall stack` | verwijdert de objecten van de chart — **niet** de PVC's van `volumeClaimTemplates` |

Een detail dat telt: een ConfigMap wijzigen herstart de Pods niet (labo 14). De chart annoteert het Pod-template met `checksum/config: {{ … | sha256sum }}`; elke wijziging van de ConfigMap verandert de annotatie, en lokt dus een rolling update uit. Het is het meest gekopieerde Helm-idioom.

> **→ GESCHIEDENIS**
> Helm ontstaat in 2015 bij Deis als "apt voor Kubernetes", gaat naar de CNCF en wordt de standaard voor publieke *charts* (databases, Prometheus, ingress controllers…). Helm 2 vereiste een servercomponent, Tiller, te machtig naar de zin van beveiligingsteams; Helm 3 (2019) schrapt hem: de client praat rechtstreeks met de API server, met jouw kubeconfig en jouw RBAC-rechten. Helm 4 (eind 2025) behoudt dezelfde CLI. Kustomize, in 2017 bij Google geboren, zit sinds 2019 in `kubectl` (`kubectl apply -k`).

> **K3S**
> Je gebruikt Helm al sinds labo 11 zonder het te weten: k3s installeert Traefik via een `HelmChart`-object (CRD) dat zijn controller `klipper-helm` toepast; `helm list -A` toont de releases `traefik` en `traefik-crd` in `kube-system`. Het binaire bestand `helm` wordt niet meegeleverd: je downloadt het, en het kent de kubeconfig van k3s niet: zonder `export KUBECONFIG=/etc/rancher/k3s/k3s.yaml` zoekt het een cluster op `localhost:8080` en faalt het.

## 3. Kustomize: patches, geen templates

Kustomize vertrekt van een **base** (geldige YAML, zo toepasbaar) en **overlays** per omgeving die ernaar verwijzen en ze wijzigen: `namespace`, `replicas`, `images` (een tag veranderen zonder de YAML aan te raken), `labels`, en `patches` (strategic merge of JSON patch). Geen `{{ }}`, geen waarden: elke overlay is op zichzelf leesbaar, en `kubectl kustomize overlays/prod` toont het exacte resultaat; `kubectl apply -k overlays/prod` past het toe. Veel teams combineren beide: `helm template … | kustomize` om een publieke chart te patchen die ze niet willen forken.

| | Helm | Kustomize |
|---|---|---|
| Model | templates + waarden | zuivere YAML + patches |
| Verspreiding | geversioneerde charts, repositories, `.tgz` | een Git-repository |
| Geschiedenis / rollback | ja (revisions) | nee (Git draagt ze) |
| Geschikt voor | software die anderen installeren (publieke charts, interne producten) | je eigen applicaties, vooral in GitOps |

## 4. Tags, digests en registry

Een **tag** is beweeglijk: `api:1.1` kan morgen naar een andere image wijzen, en `latest` is een belofte van verrassingen. Twee bedrijfspraktijken: **onveranderlijke** tags (de Git-SHA of een SemVer die nooit herschreven wordt, gezet door de CI), en, voor wat in productie draait, de **digest**: `image: localhost:5000/api@sha256:b82020cb…` wijst exact één manifest aan, voor altijd. `podman push --digestfile` geeft de digest zoals de registry hem geregistreerd heeft — anders dan `podman image inspect --format '{{.Digest}}'` op de lokale image, want de push hercomprimeert de lagen; en `kubectl get pod -o jsonpath='{.status.containerStatuses[0].imageID}'` toont degene die de node werkelijk gepulld heeft.

Tot nu toe gingen de images via `podman save | sudo k3s ctr images import`. De echte weg is een **registry**: de CI pusht, de cluster pullt. De `registry:2` van labo 10 volstaat: k3s pullt `localhost:5000/api:1.1` over HTTP zonder configuratie (containerd tolereert `localhost` onversleuteld). Voor een bedrijfsregistry (interne TLS, inloggegevens, mirror van Docker Hub) leest k3s `/etc/rancher/k3s/registries.yaml` (`mirrors:` en `configs:` met `auth:` / `tls:`) en moet je `sudo systemctl restart k3s` uitvoeren. Op elke andere cluster gaan de inloggegevens in een Secret `kubernetes.io/dockerconfigjson`, waarnaar `imagePullSecrets` verwijst.

> **→ BEVEILIGING**
> Een beweeglijke tag is een lek in de reproduceerbaarheid en een toegangspoort (wie mag `api:1.1` pushen?). De digest sluit die poort; `imagePullPolicy: Always` met een beweeglijke tag verschuift ze alleen. Serieuze ketens ondertekenen images (cosign) en weigeren bij admission (Kyverno, Gatekeeper) wat niet ondertekend of niet gescand is. En het wachtwoord van de registry staat nooit in de chart: `--set` vanuit de kluis van de CI, of een extern Secret (labo 14).

## 5. CI/CD → cluster, en GitOps

De keten van labo 10 krijgt er een stap bij:

```
deploy:                                  # fragment van een pipeline (GitLab CI, GitHub
Actions: zelfde idee)
  script:
    - podman build -t registry.intern/api:$CI_COMMIT_SHA .
    - podman push registry.intern/api:$CI_COMMIT_SHA
    - helm upgrade --install api ./chart -n facturatie-prod -f values-prod.yaml
        --set api.image.tag=$CI_COMMIT_SHA --wait --timeout 5m
```

De *runner* heeft een kubeconfig beperkt tot een ServiceAccount (labo 16) met `edit` op **één** namespace; `--wait` laat de job falen als de rollout niet convergeert, en een `helm rollback` is het vervolg van de pipeline bij mislukking. **GitOps** keert de richting om: niemand voert `helm upgrade`

uit; een Git-repository beschrijft de gewenste toestand (charts + values, of Kustomize-overlays), en een agent in de cluster (**Argo CD**, **Flux**) leest ze voortdurend en reconcilieert — de lus van labo 11, een verdieping hoger. De CI doet niets meer dan bouwen, pushen en **een tag in Git wijzigen**; geschiedenis, review en rollback zijn die van Git.

## 6. k3s tegenover bedrijfsclusters

| | **k3s (wat je hebt)** | **EKS / AKS / GKE** | **OpenShift** |
|---|---|---|---|
| Control plane | één binair bestand, bij jou | beheerd door de cloud, gefactureerd | beheerd (OpenShift) of geïnstalleerd |
| `Service: LoadBalancer` | ServiceLB (poorten van de node) | cloud load balancer, publiek IP | idem + **Route** (zijn eigen Ingress) |
| StorageClass | `local-path` (schijf van de node) | netwerkschijven (EBS, Azure Disk, PD) | idem |
| Ingress | Traefik voorgeïnstalleerd | te installeren (nginx, ALB…) | ingebouwde Router |
| Identiteit | kubeconfig = `cluster-admin` | IAM / Entra / Google → RBAC | ingebouwde OAuth, SCC (strikte beveiliging, geen root) |
| CLI | `kubectl` | `kubectl` | `oc` (superset) |

Al de rest — Pod, Deployment, Service, ConfigMap, PVC, RBAC, Helm — is **identiek**: dat is het nut van een standaard-API. Wat verandert zit aan de rand: wie de load balancer, de schijf en de identiteit levert, en wat dat kost.

## 7. In het bedrijf

- **Eén chart per applicatie**, samen met haar geversioneerd; de `values-<env>.yaml` in dezelfde repository, de secrets elders.
- **Onveranderlijke tags gezet door de CI**, digests in productie, `latest` verboden.
- **De CI-runner heeft een rol, geen `cluster-admin`**; beter nog: GitOps, en de CI raakt de cluster niet meer aan.
- **Helm voor wat je installeert, Kustomize voor wat je schrijft** — of beide, zonder dogma.
- **k3s is een ontwikkelcluster** die trouw is aan de API; wat je erop geleerd hebt, neem je ongewijzigd mee.

---

## Onthouden

- Chart = `Chart.yaml` + `values.yaml` + `templates/`; release = geïnstalleerde chart; revision = elke `install/upgrade/rollback`, opgeslagen in een Secret.
- `helm upgrade … --set` / `-f values-prod.yaml`, `helm history`, `helm rollback 1` (dat een revision maakt); `uninstall` bewaart de PVC's.
- `checksum/config` op het Pod-template: een gewijzigde ConfigMap herdeployt.
- Kustomize = base + overlays (`namespace`, `replicas`, `images`, `patches`), `kubectl apply -k`, geen enkel template.

- Beweeglijke tag ≠ digest: `--digestfile` bij de push, `imageID` aan de kant van de node, `image@sha256:` in productie.
- k3s pullt `localhost:5000` zonder configuratie; een bedrijfsregistry gaat via `registries.yaml` of `imagePullSecrets`.
- CI: build → push → `helm upgrade --install --wait` met een beperkte rol; GitOps: Git beschrijft, Argo CD/Flux reconcilieert.

## Woordenlijst

**chart**: Helm-pakket (templates + waarden). — **release / revision**: chart geïnstalleerd onder een naam / genummerde versie van die installatie. — **values**: parameters van de chart, overschreven met `-f` of `--set`. — **`helm template`**: rendering zonder installatie. — **Kustomize**: base + overlays, patches zonder templates. — **overlay**: variant per omgeving. — **digest**: onveranderlijke vingerafdruk van een image (`@sha256:`). — **registries.yaml**: registry-configuratie van k3s. — **imagePullSecrets**: registry-inloggegevens van een Pod. — **GitOps**: Git als bron van waarheid, gereconcilieerd door een agent (Argo CD, Flux). — **beheerde cluster**: control plane geleverd door een cloud (EKS, AKS, GKE). — **OpenShift**: Kubernetes-distributie van Red Hat (Routes, SCC, `oc`).